عوالم البرمجة

أكاديمية عوالم البرمجة

Git & GitHub 101

من الصفر إلى أول Pull Request

.BEYOND CODE. BEYOND LIMITS

01

الوحدة 01

فهم Git وGitHub

ما هو التحكم بالإصدارات، ولماذا يستخدمه كل مطوّر، وكيف تعمل أدوات Git وGitHub وVS Code معًا.

ما هو التحكم بالإصدارات؟

What is version control?

أهداف التعلم

- أن تشرح بكلماتك ما هو التحكم بالإصدارات.
- أن تذكر ثلاث مشكلات تسببها نسخ مجلدات المشروع يدويًا.
- أن تفهم كيف يسجّل Git التغييرات كسلسلة من الـ commits.
- أن تفرّق بين التحكم بالإصدارات المحلي والموزّع.

المتطلبات المسبقة

لا شيء. تحتاج فقط إلى حاسوب ومشروع، ولو ملفًا نصيًا بسيطًا، عدّلته أكثر من مرة.

مقدمة

تخيّل أنك تكتب تقريرًا مهمًا. يوم الاثنين تحفظ `report.docx`. يوم الثلاثاء تعدّله وتحفظ `report_final.docx`. وبحلول الجمعة يصبح عندك `report_final2.docx` و `report_REAL_final.docx`، ولم تعد متأكدًا أيّها الصحيح. التحكم بالإصدارات يحل هذه المشكلة بالضبط، ويفعل ذلك لكل ملفات المشروع دفعة واحدة.

الشرح

التحكم بالإصدارات (Version Control) نظام يسجّل كل تغيير يحدث على مجموعة ملفات: من أجراه، ومتى، ولماذا. تستطيع العودة إلى أي لحظة سابقة، ومقارنة لحظتين، والعمل مع آخرين دون أن يمحو أحدكم عمل الآخر.

لماذا يحتاجه المطوّرون؟ الكود يتغيّر باستمرار، وقد تُفسد ميزة جديدة شيئًا كان يعمل بالأمس. مع التحكم بالإصدارات ترى بالضبط ما الذي تغيّر وتتراجع عنه بأمان. والفرق أيضًا تحتاجه: يستطيع خمسة أشخاص تعديل المشروع نفسه ودمج أعمالهم.

مشكلة نسخ المجلدات. النسخ تستهلك مساحة، والأسماء تصبح مربكة، ولا شيء يشرح *لماذا* وُجدت نسخة ما، وشخصان ينسخان المجلد نفسه لا يمكن دمج تعديلاتهما تلقائيًا.

كيف يعمل Git؟ لا يحتفظ Git بنسخ عشوائية. عندما تطلب منه، يحفظ **commit**: لقطة من مشروعك مع رسالة واسم صاحبها. وكل commit يشير إلى الذي قبله، فيصبح تاريخ مشروعك سلسلة تستطيع المشي فيها إلى الخلف.

محلي وموزّع. الأنظمة القديمة كانت تحفظ التاريخ في خادم مركزي واحد. أما Git فهو **موزّع**: لدى كل مطوّر التاريخ كاملًا على جهازه. يمكنك العمل دون إنترنت، وإن ضاع الخادم فأي نسخة تستطيع استعادته.

تشبيه من الحياة

فكّر بلعبة فيديو فيها خانات حفظ. كل حفظ لحظة تستطيع العودة إليها. والـ commit نقطة حفظ لمشروعك، والرسالة هي الاسم الذي تكتبه عليها، مثل «قبل المعركة الأخيرة».

مخطط توضيحي



الشكل 1 · نسخ المجلدات مقابل سجل Git

الأوامر

لا توجد أوامر في هذا الدرس. في الدرس 2.1 ستثبّت Git وتشغّل أول أمر وهو `git --version` .

أخطاء شائعة وحلولها

«Git أداة نسخ احتياطي.»	Git يسجّل التغييرات ذات المعنى التي تختار حفظها. احتفظ بنسخ احتياطية حقيقية أيضًا.
«يجب أن أكون متصلًا بالإنترنت لأستخدم Git.»	Git يعمل دون إنترنت تمامًا. تحتاج الإنترنت فقط للمشاركة عبر GitHub.
«التحكم بالإصدارات للفرق الكبيرة فقط.»	يفيد الشخص الواحد منذ اليوم الأول للمشروع.

تمرين عملي

اختر مشروعًا أو مستندًا عدّلته مرات كثيرة. اكتب ثلاث حالات تمنّيت فيها العودة إلى نسخة سابقة أو مقارنة النسخ. وبجانب كل حالة اكتب أي فكرة من Git تفيدك (التاريخ، أو الـ commit، أو المشاركة).

النتيجة المتوقعة

قائمة قصيرة من ثلاث حالات حقيقية، كل واحدة مرتبطة بفكرة من هذا الدرس. لا توجد قائمة صحيحة وأخرى خاطئة. هذا التمرين يهيّئك للدرس 1.2.

اختبر فهمك

1. ماذا يخزّن الـ commit؟

- ○ نسخة من الإنترنت
- ○ لقطة من المشروع ورسالة واسم صاحبها
- ○ أسماء الملفات المتغيرة فقط

2. لماذا يسمى Git موزّعًا؟

- ○ لأن لدى كل مطوّر التاريخ كاملًا على جهازه
- ○ لأنه يعمل على خادم واحد فقط
- ○ لأنه يوزّع ملفات مجانية

3. ما مشكلة نسخ المجلدات يدويًا؟

- ○ ويندوز يحذف النسخ
- ○ أسماء غير واضحة ولا تفسير للتغييرات
- ○ تصبح الملفات للقراءة فقط

الإجابات

- لقطة من المشروع ورسالة واسم صاحبها — الـ commit لقطة مع رسالة ومؤلف، ومرتبطة بالـ commit الذي قبلها.
- لأن لدى كل مطوّر التاريخ كاملًا على جهازه — كل نسخة تحتوي التاريخ كاملًا، فلا يعتمد العمل على خادم مركزي واحد.
- أسماء غير واضحة ولا تفسير للتغييرات — أسماء مثل final2 لا تقول ما الذي تغيّر ولماذا، ولا يمكن دمج النسخ تلقائيًا.

الخلاصة

التحكم بالإصدارات يسجّل تاريخ المشروع. يحفظ Git لقطات تسمى commits، ويُبقي التاريخ كاملًا على كل جهاز، ويسمح لك بالعودة إلى أي نقطة أو دمج عمل عدة أشخاص.

مصطلحات أساسية

التحكم بالإصدارات (Version control)	نظام يسجّل تغييرات الملفات مع الزمن
Commit	لقطة محفوظة مع رسالة واسم صاحبها
التاريخ (History)	سلسلة الـ commits في المشروع
موزّع (Distributed)	كل نسخة تحتوي التاريخ كاملًا

الدرس التالي

الدرس 1.2 — Git مقابل GitHub مقابل VS Code. عرفت الآن ما يفعله Git. بعده ستتعرّف على الأدوات الثلاث التي ستستخدمها وكيف تعمل معًا.

Git مقابل GitHub مقابل VS Code

Git vs GitHub vs VS Code

أهداف التعلم

- أن تعرّف Git وGitHub وVS Code بجملة لكل منها.
- أن تشرح المستودع المحلي والمستودع البعيد.
- أن تفرّق بين المستودع العام والخاص.

المتطلبات المسبقة

الدرس 1.1.

مقدمة

يستخدم المبتدئون كلمتي Git وGitHub وكأنهما شيء واحد. هما في الحقيقة أداتان مختلفتان، وVS Code ثالثتهما. ومعرفة الفرق تزيل معظم الحيرة في البداية.

الشرح

Git برنامج مجاني على جهازك يسجّل تاريخ ملفاتك، ويعمل دون إنترنت.

GitHub موقع يحفظ مشاريع Git على الإنترنت، ويضيف المشاركة والعمل الجماعي: نسخًا احتياطية، وقضايا (Issues)، وطلبات دمج (Pull Requests)، وصفحة تعريفية.

VS Code (Visual Studio Code) محرر أكواد فيه Git مدمج، فتستطيع التجهيز والحفظ والرفع بأزرار بدل الكتابة.

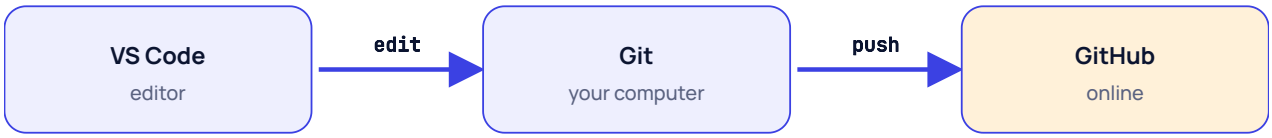
المستودع (Repository) هو مجلد المشروع مع تاريخ Git. نسختك هي **المستودع المحلي**، والنسخة على GitHub هي **المستودع البعيد**. وال**commit** لقطة محفوظة واحدة، و**الفرع (branch)** خط من الـ commits. ستدرس الفروع في الوحدة 7.

المستودعات **العامة** يراها أي شخص، و**الخاصة** لا يراها إلا أنت ومن تدعوهم. لا تضع كلمات السر في أي منهما أبدًا.

تشبيه من الحياة

تأليف كتاب: VS Code طاولة الكتابة، وGit المحرّر الذي يحتفظ بكل مسودة، وGitHub المجلد المشترك على السحابة الذي يقرؤه فريقك.

مخطط توضيحي



الشكل 1 · كيف تعمل الأدوات الثلاث معًا

الأوامر

لا توجد أوامر بعد. في الدرس 2.1 ستثبّت الأدوات وتشغّل `git --version` .

أخطاء شائعة وحلولها

الظن أن GitHub ضروري لاستخدام Git.	Git يعمل وحده. GitHub اختياري لكنه مفيد جدًا للمشاركة.
الظن أن VS Code وGit شيء واحد.	VS Code محرر فقط، ويعطيك أزرارًا لاستخدام Git.
جعل المستودع عامًا بالخطأ.	تحقّق من إعداد الخصوصية قبل إنشاء المستودع أو نشره.

تمرين عملي

افتح github.com في المتصفح، ثم افتح أي مستودع عام. جد ثلاثة أشياء: نص README، وقائمة الملفات، واسم الفرع (غالبًا `main`).

النتيجة المتوقعة

تستطيع الإشارة إلى README وقائمة الملفات ومحدد الفرع في الصفحة. لم تحتج إلى حساب لهذا.

اختبر فهمك

1. أي أداة تحفظ مشروعك على الإنترنت؟

- ○ Windows Terminal
- ○ GitHub
- ○ VS Code

2. أين يوجد المستودع المحلي؟

- ○ على جهازك
- ○ على GitHub فقط
- ○ داخل المتصفح

3. من يرى المستودع الخاص؟

- ○ موظفو GitHub فقط
- ○ أنت ومن تدعوهم فقط
- ○ الجميع

الإجابات

- GitHub — GitHub يستضيف المستودعات البعيدة.
- على جهازك — المستودع المحلي هو تاريخ المشروع المحفوظ على جهازك.
- أنت ومن تدعوهم فقط — الخاص يعني أنه مرئي لك وللمتعاونين الذين تختارهم.

الخلاصة

Git يتتبّع التغييرات، وGitHub يستضيف المستودعات على الإنترنت، وVS Code محرر فيه أزرار Git. جهازك يحمل المستودع المحلي وGitHub يحمل البعيد.

مصطلحات أساسية

المستودع (Repository)	مجلد مشروع مع تاريخ Git
محلي / بعيد	على جهازك / على GitHub
عام / خاص	مرئي للجميع / للمدعوين فقط
الفرع (Branch)	خط من الـ commits

الدرس التالي

الدرس 1.3 — سير عمل Git. بعده ستتبع ملفًا من محررك حتى GitHub مرحلة بمرحلة.

فهم سير عمل Git

Understanding the Git workflow

أهداف التعلم

- أن تسمّي الأماكن الأربعة التي قد يوجد فيها التغيير.
- أن تميّز الملفات غير المتتبَّعة والمعدّلة والمجهّزة والمحفوظة.
- أن تشرح add ثم commit ثم push.

المتطلبات المسبقة

الدرسان 1.1 و1.2.

مقدمة

لا يحفظ Git كل شيء تلقائيًا. أنت تختار ماذا تحفظ ومتى. وهذا الاختيار يحدث على مراحل، وتعلّمها يجعل كل أمر لاحق سهل الفهم.

الشرح

مجلد العمل (Working directory): مجلد مشروعك حيث تعدّل الملفات.

الملفات **غير المتتبَّعة (Untracked)** ملفات جديدة لم يرها Git بعد. والملفات **المعدّلة (Modified)** ملفات متتبَّعة غيّرتها منذ آخر commit.

منطقة التجهيز (Staging area): غرفة انتظار. تضع فيها التغييرات التي تريدها في الـ commit القادم. الأمر هو `git add`.

المستودع المحلي: التاريخ على جهازك. الأمر `git commit` يحفظ التغييرات المجهّزة كـ commit جديد.

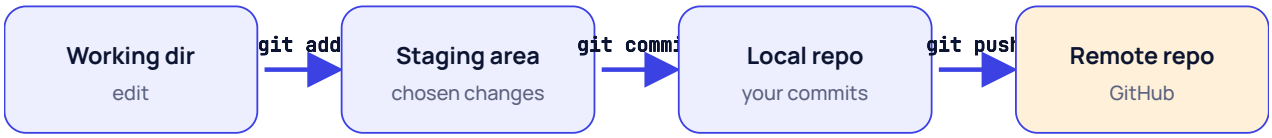
المستودع البعيد: النسخة على GitHub. الأمر `git push` يرسل إليه الـ commits الجديدة.

سجل الـ commits هو قائمة كل الـ commits. **تذكّر:** التجهيز ليس حفظًا، والحفظ ليس رفعًا. كل منهما خطوة مستقلة.

تشبيه من الحياة

تجهيز طرد بريدي: تعدّل في غرفتك (مجلد العمل)، وتضع ما اخترته في الصندوق (التجهيز)، وتغلقه وتكتب عليه (commit)، ثم ترسله بالبريد (push).

مخطط توضيحي



الشكل 1 · سير عمل add ثم commit ثم push

الأوامر

ستتعلم هذه الأوامر في الوحدتين 4 و5. الآن يكفي أن تتعرف عليها: `git add` و `git commit -m "message"` و `git push` .

أخطاء شائعة وحلولها

الظن أن add يحفظ العمل.	هو يجهّز التغييرات فقط. لا يُحفظ شيء حتى تنفّذ commit.
الظن أن commit يرفع إلى GitHub.	الـ commit يحفظ على جهازك فقط. الرفع يتم بـ push.
توقع أن تُتتبَّع الملفات الجديدة تلقائيًا.	الملفات الجديدة غير متتبَّعة حتى تضيفها بـ add.

تمرين عملي

تخيّل أنك عدّلت `index.html` ، ثم نفّذت `git add index.html` ، ثم `git commit -m "Add homepage"` . اكتب في أي مرحلة يكون التغيير بعد كل إجراء من الثلاثة.

النتيجة المتوقعة

بعد التعديل: معدّل في مجلد العمل. بعد add: مجهّز. بعد commit: محفوظ في المستودع المحلي ولم يصل GitHub بعد.

اختبر فهمك

1. أي أمر يرسل الـ commits إلى GitHub؟

- ○ git commit
- ○ git push
- ○ git add

2. الملف الجديد الذي لم يره Git يسمى...

- ○ غير متتبَّع (Untracked)
- ○ محفوظ (Committed)
- ○ مرفوع (Pushed)

3. هل ينشئ git add عملية commit؟

- ○ فقط يوم الجمعة
- ○ لا، هو يجهّز التغييرات فقط
- ○ نعم

الإجابات

- git push — push يرفع الـ commits إلى المستودع البعيد.
- غير متتبَّع (Untracked) — الملفات الجديدة غير متتبَّعة حتى تضيفها بـ add.
- لا، هو يجهّز التغييرات فقط — التجهيز يحضّر التغييرات، والـ commit هو الذي ينشئ اللقطة.

الخلاصة

ينتقل التغيير من مجلد العمل إلى منطقة التجهيز (add)، ثم إلى المستودع المحلي (commit)، ثم إلى المستودع البعيد (push).

مصطلحات أساسية

مجلد العمل	مجلد مشروعك
منطقة التجهيز	حيث تنتظر التغييرات الـ commit القادم
غير متتبَّع / معدّل	ملف جديد / ملف متتبَّع تغيّر
Push	إرسال الـ commits إلى المستودع البعيد

الدرس التالي

الوحدة 2 — تثبيت Git وإعداده. عرفت النظرية. الآن ستثبّت الأدوات وتشغّل أول أمر.

المختبر العملي 1 · حدّد كل مرحلة

اقرأ السيناريو واملأ الجدول. السيناريو: تنشئ `style.css` ، وتعدّل `index.html` ، ثم تنفّذ `git add index.html` و `git commit -m "Add homepage"` و `git push` .

اللحظة	أين تغيير `index.html` ؟	أين `style.css` ؟
بعد التعديل	معدّل (مجلد العمل)	غير متتبَّع
بعد git add	مجهّز	غير متتبَّع
بعد git commit	المستودع المحلي	غير متتبَّع
بعد git push	المستودع البعيد (GitHub)	غير متتبَّع

معيار التقييم: 4 صفوف صحيحة = نجاح. لاحظ أن `style.css` يبقى غير متتبَّع لأنك لم تضفه بـ add.

اختبار الوحدة 1 (علامة النجاح: 70%)

1. ما هو Git؟

- ○ موقع إلكتروني
- ○ محرر أكواد
- ○ برنامج يتتبّع تاريخ الملفات

2. ما هو GitHub؟

- ○ صيغة ملفات
- ○ موقع يستضيف مستودعات Git
- ○ طرفية

3. ماذا تفعل عملية التجهيز (staging)؟

- ○ تحضّر التغييرات للـ commit القادم
- ○ ترفع إلى GitHub
- ○ تحذف الملفات

4. بعد git commit أين توجد تغييراتك؟

- ○ لا مكان
- ○ في المستودع المحلي
- ○ على GitHub

5. هل يعمل Git دون إنترنت؟

- ○ لا
- ○ فقط على ويندوز
- ○ نعم

الإجابات

- برنامج يتتبّع تاريخ الملفات — Git هو برنامج التحكم بالإصدارات على جهازك.
- موقع يستضيف مستودعات GitHub — Git يحفظ المستودعات البعيدة على الإنترنت.
- تحضّر التغييرات للـ commit القادم — التجهيز يحدد التغييرات التي ستدخل الـ commit القادم.
- في المستودع المحلي — الـ commit يحفظ على جهازك، والرفع خطوة منفصلة.
- نعم — Git موزّع، والتاريخ محلي.